Reason. Plan. Act. Physical AI Workshop

Lab 3.1: State estimation

Guy Azran and Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology



Where we are

Guy Azran and Sarah Keren

- So far, `get_block_positions()` read the simulator's memory: a God's-eye view
- Joint encoders measure the robot itself (**proprioceptive**). To know the world, it needs an **exteroceptive** sensor: a camera
- **State estimation**: from what the sensors say to what the planners need

$$\text{image} \xrightarrow{\text{detection}} \text{pixels of each block} \xrightarrow{\text{camera model}} \text{3D positions}$$

$$\xrightarrow{\text{identification}} \text{position of each named block} \xrightarrow{\text{predicates}} \text{facts}$$

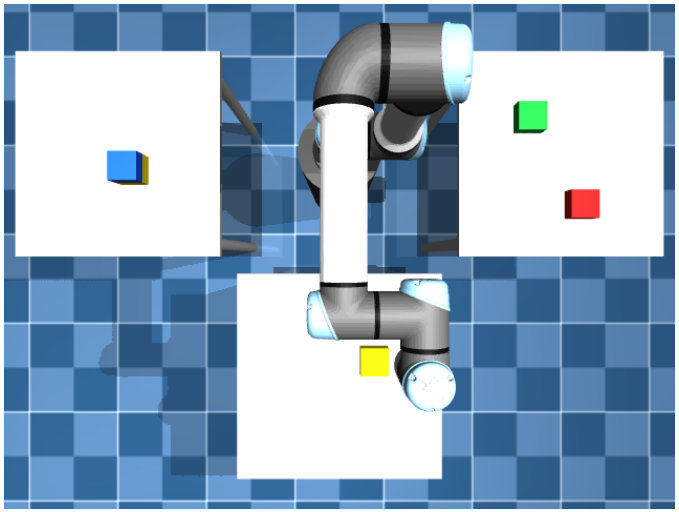
1.1 Task planning	2.1 Motion planning	**3.1 State estimation**	4.1 Integrated TAMP
1.2 Inverse kinematics	2.2 Control	3.2 Decoupled TAMP	4.2 VLAs

Today

Learn	An RGB-D camera, and finding blocks by their color
Do	Task 1: color masks
Do	Task 2: from pixels to blobs
Learn	The pinhole camera model
Do	Task 3: from pixels to positions in the world
Learn	From positions to facts: tracking, object permanence, predicates
Do	Task 4: the abstract state
Do	Task 5: estimate, act, estimate again
Learn	Sensor noise, and beliefs instead of facts
Do	Task 6: a belief over facts
Learn	A learned detector that we prompt in words
Do	Task 7: plug it into the estimator

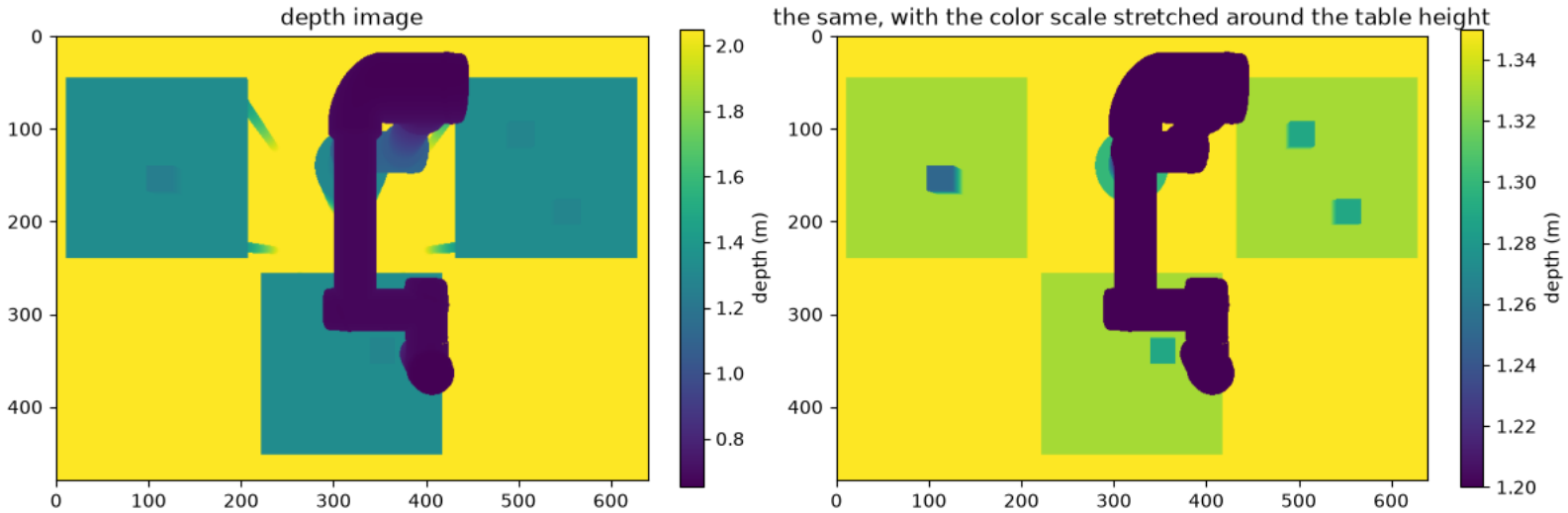
A camera on the ceiling

Guy Azran and
Sarah Keren



An **RGB-D** camera gives a color image and, for every pixel, the **depth** of the surface it shows.

How many blocks do you see?

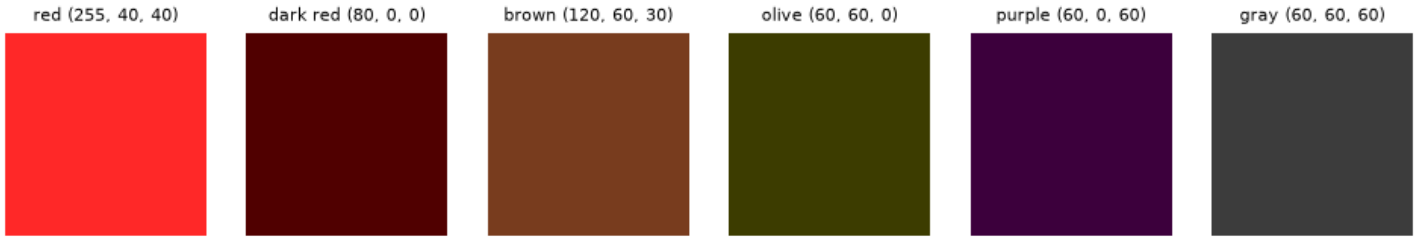


Which image tells you that the stack is two blocks high?

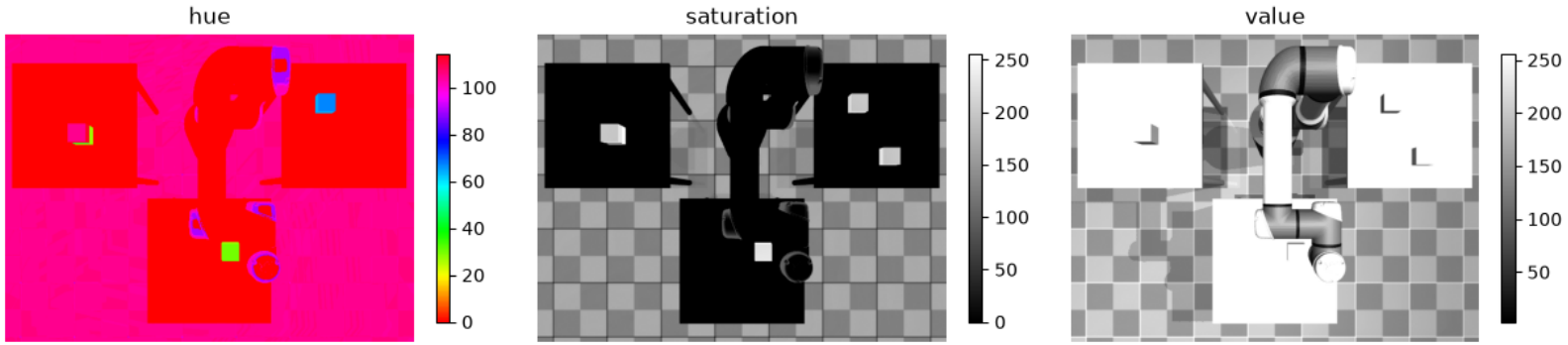
Finding the blocks by their color

Guy Azran and
Sarah Keren

No pixel has the red block's exact color, $(255, 26, 26)$: what the camera sees depends on the light. A box in RGB also accepts these as "red":



HSV separates the color (hue) from how pure and how bright it is:



The floor has the blue block's hue. Saturation and value keep it

Task 1: color masks

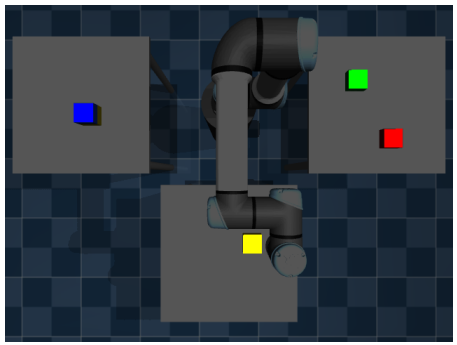
Guy Azran and Sarah Keren

⚒ **Task 1**: Write `color_mask(rgb, color)`, which returns a boolean image that is `True` at the pixels of the given color. Use `cv2.inRange(hsv, low, high)` with the hue ranges below (in OpenCV's 0 to 179 units), a minimum saturation of 150 and a minimum value of 200.

```
HUE_RANGES = {"yellow": [(20, 40)], "green": [(50, 80)], "blue":
[(100, 130)], "red": [(0, 10), (170, 180)]}
MIN_SATURATION, MIN_VALUE = 150, 200


def color_mask(rgb, color):
    """A boolean image: True at the pixels whose color is the
named one."""
    return np.zeros(rgb.shape[:2], dtype=bool)  # your code here
```

- `cv2.cvtColor(rgb, cv2.COLOR_RGB2HSV)`
- Red needs two ranges: its hue wraps around from 179 to 0



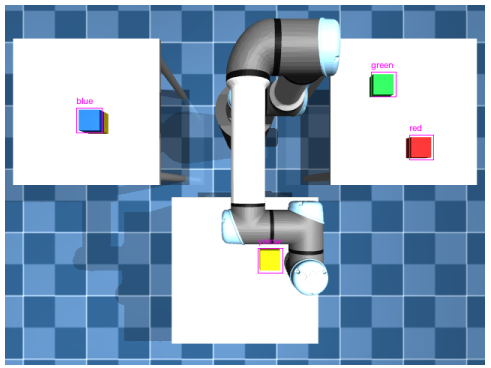
Task 2: from pixels to blobs

Guy Azran and Sarah Keren

⚒ **Task 2**: Write `find_blobs(mask, min_area)`, which returns one dictionary per blob with at least `min_area` pixels, with keys `pixels` (a boolean image of the blob), `centroid` (`(u, v)`, in pixels), `area` and `box` (`(left, top, width, height)`). Note that `connectedComponentsWithStats` numbers the background as component 0. `detect` then runs it on the mask of every color and adds the color to each detection.

```python
def find_blobs(mask, min_area=300):
    """The connected blobs of the mask with at least min_area pixels."""
    return []  # your code here
```

- **Connected components**: pixels that touch form one blob
- Tiny blobs are noise, or a mostly hidden block



What happens with `min_area=50`?

The pinhole camera

Guy Azran and Sarah Keren

Extrinsics: where the camera is ($\mathbf{t}$) and where it looks (R). MuJoCo cameras look along $-z$.

$$\mathbf{p}_{\text{cam}} = R^{\top}(\mathbf{p} - \mathbf{t}), \qquad d = -z_{\text{cam}}$$

Intrinsics: focal length and principal point. Here $f = \frac{h/2}{\tan(\text{fovy}/2)} = 433$ px.

$$u = c_x + f_x \frac{x_{\text{cam}}}{d}, \qquad v = c_y - f_y \frac{y_{\text{cam}}}{d}$$

Projection loses the depth: every point on a ray lands on the same pixel. The depth image gives it back:

$$x_{\text{cam}} = \frac{(u - c_x)\, d}{f_x}, \qquad y_{\text{cam}} = -\frac{(v - c_y)\, d}{f_y}, \qquad z_{\text{cam}} = -d$$

$$\mathbf{p} = \mathbf{t} + R\, \mathbf{p}_{\text{cam}}$$

Task 3: from pixels to positions in the world

Guy Azran and Sarah Keren

⚒ **Task 3**: Write `pixels_to_world(u, v, depth_values, camera)`, which takes arrays of pixel coordinates and their depths and returns an array of shape `(n, 3)` of world points. Then `localize(detection, depth)`: the center of the block a detection shows.

```python
def pixels_to_world(u, v, depth_values, camera=CAMERA):
    """World coordinates (n x 3) of the pixels (u, v) (arrays of
length n) that show surfaces at the given depths."""
    return np.zeros((len(u), 3))  # your code here


def localize(detection, depth, camera=CAMERA):
    """The position of the center of the block a detection shows
(world frame)."""
    return np.zeros(3)  # your code here
```

- Take the *median* of the blob's points: stray pixels at the edge belong to the table or a neighbor
- The camera sees the *top face*, half a block above the center

Which block is which?

Guy Azran and
Sarah Keren



`b2` hides `b1`

Colors are not names: two blocks are yellow.

- **Tracking**: objects do not jump. Match each detection to the nearest block of its color
- **Object permanence**: a block we do not see is where we saw it last
- **The gripper's sensor**: a held block moves with the tool, and `release` logs where it was let go

The estimator gets the true positions once, at the start. From then on it is on its own.

From positions to facts

Guy Azran and Sarah Keren

Every predicate needs a definition in terms of things we can measure:

fact	holds when
`(ontable ?b ?t)`	the center is above the table, half a block above its top
`(on ?b1 ?b2)`	`?b1` is right above `?b2`, one block higher
`(clear ?b)`	no block is on `?b`
`(holding ?b)`	the gripper's sensor says so
`(handempty)`	

Every "above" needs a **tolerance**: here half a block height (2 cm) and half a block width (4 cm). What goes wrong if it is too tight? Too loose?

Task 4: the abstract state

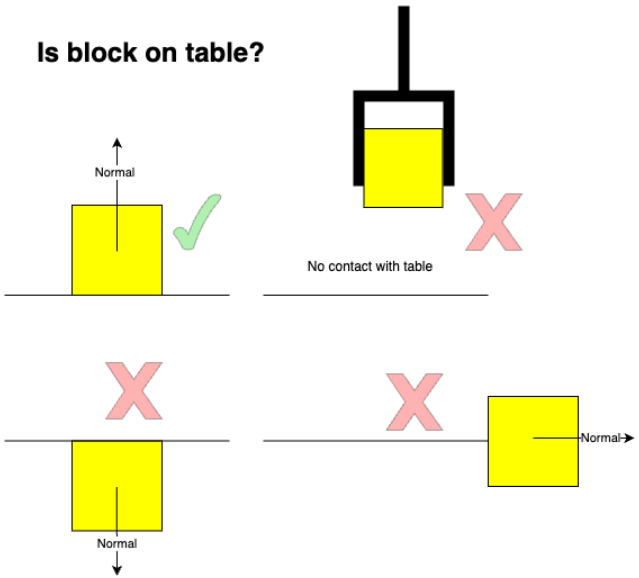
Guy Azran and
Sarah Keren

⚒ **Task 4**: Write `abstract_state(positions, held)`, which returns the set of facts that hold, each as a tuple like `("on", "b2", "b1")`, `("ontable", "b1", "t1")`, `("clear", "b3")`, `("holding", "b2")` or `("handempty",)`. A held block is on nothing and nothing is on it.

```python
def abstract_state(positions, held=None):
    """The facts of the task planner that hold given the positions
of the blocks and the block in the gripper."""
    facts = set()
    # your code here
    return facts
```

- A held block is on nothing, and nothing is on it
- Expect the initial state of the Lab 1.1 problem, including `(on b2 b1)` for a block the camera cannot see

Checking the estimator: physics as ground truth



To *check* the estimator, we may cheat:

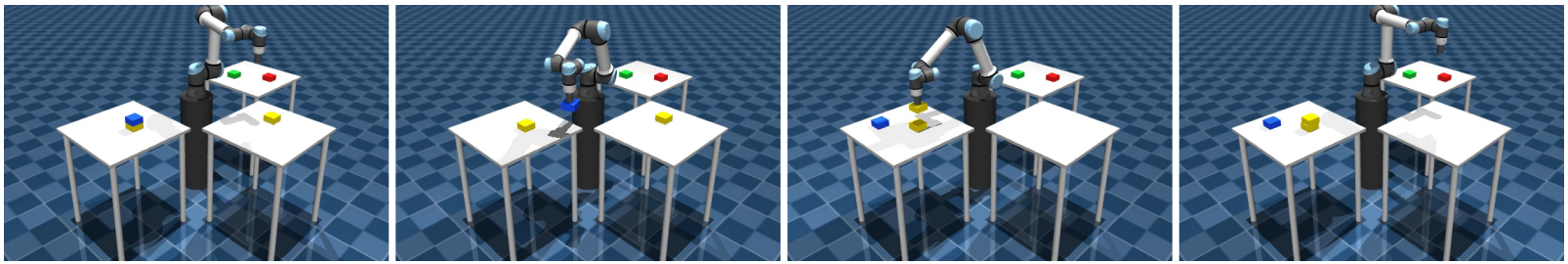
- MuJoCo computes the contacts between bodies
- A block is on whatever pushes it *up*
- Contacts exist only once the physics runs: `look()` moves the arm away, lets the scene settle, and observes

Task 5: estimate, act, estimate again

Guy Azran and
Sarah Keren

⚒ **Task 5**: Execute the plan of tutorial 1.1 (`unstack(b2, b1)` , `put-down(b2, t1)` , `pick-up(b3, t2)` , `stack(b3, b1)`), as in the previous tutorial. After every action, `look` (adding its frames to the video), update the estimator, and compare the estimated state with the true one. Which facts change after each action?

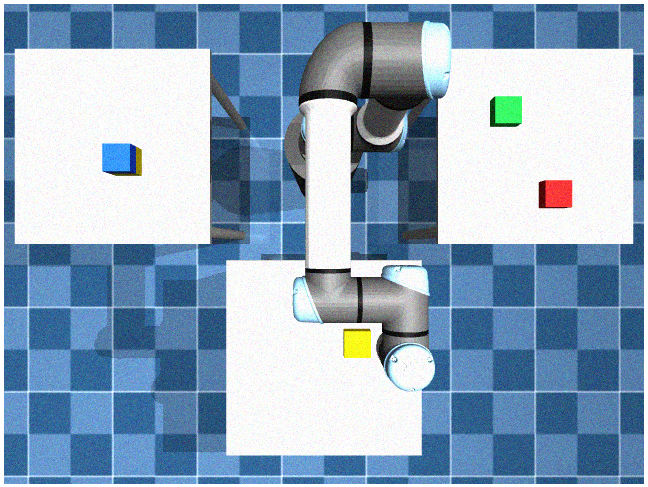
```
set_seed(42)
reset()
estimator = StateEstimator(get_block_positions())
# your code here
```



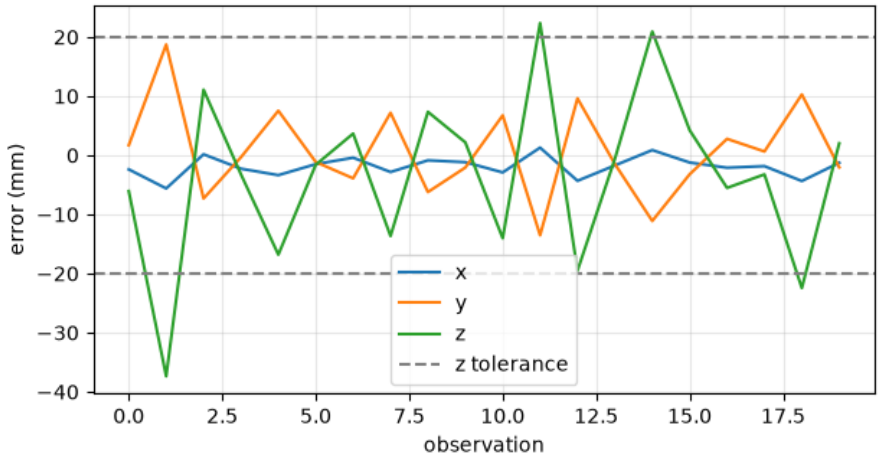
At the end, the camera sees four blocks. The robot knows about five.

Noise

Guy Azran and
Sarah Keren



color noise, depth noise, depth drift



the red block, localized in 20 noisy frames

- Per-pixel noise averages out over a blob. The per-frame drift moves all pixels together, and does not
- When the error in z exceeds the tolerance, a fact flickers off, or a wrong one flickers on

Task 6: a belief over facts

Guy Azran and Sarah Keren

⚒ **Task 6**: Write `belief(estimator, n_frames)`, which takes `n_frames` noisy observations (updating the estimator each time), computes the abstract state after each, and returns a dictionary from each fact that held at least once to the fraction of frames in which it held. The cell prints the facts sorted by probability.

```
def belief(estimator, n_frames=20):
    """The probability of every fact, estimated as its frequency
over n_frames noisy observations."""
    return {}  # your code here
```

- A planner cannot use a fact that is true in one frame and false in the next
- A **belief**: a probability for every fact. The simplest one is a frequency
- Which facts come out certain, and why?

What to do with a belief

Guy Azran and
Sarah Keren

Filter

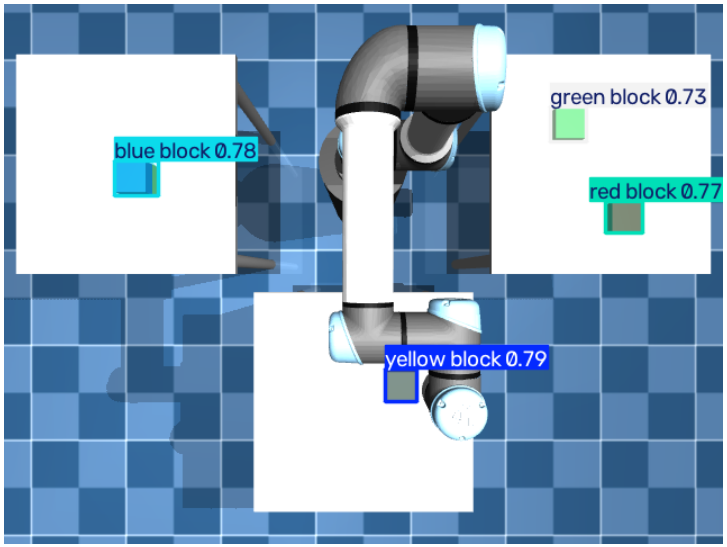
- Average the positions over the last n frames: the drift averages out, and the error shrinks as $1/\sqrt{n}$
- The **Kalman filter** weighs each observation by its uncertainty, and handles moving objects
- The **particle filter** handles beliefs like "in the cup, *or* on the floor"

Reason with it

- Look again, from closer or from another angle, before acting
- Plan in a way that works either way: planning under uncertainty

Learning to see

Guy Azran and Sarah Keren



YOLOE, prompted with four phrases

- Our detector is four hue ranges: change the light or the table color, and it breaks
- YOLO: 80 fixed classes, and “block” is not one of them
- **Open vocabulary**: trained with language, so classes are described in words: `"yellow block"`

Try other prompts: `["cube"]`, `["block"]`, `["robot arm", "table"]`.

Mind the channels: OpenCV and `ultralytics` use BGR, not RGB.

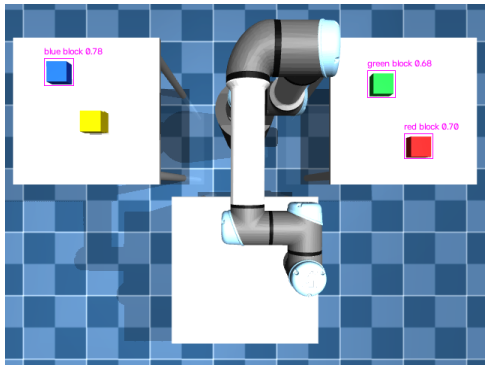
Task 7: plug it into the estimator

Guy Azran and Sarah Keren

⚒ **Task 7**: Write `detect_learned(rgb)`, which runs the detector and returns detections in the same format as `detect` (`color`, `pixels`, `centroid`, `area`, `box`), so that everything we built, `localize`, the `StateEstimator` and `abstract_state`, works unchanged.

```python
def detect_learned(rgb, conf=0.3):
    """Detection with YOLOE: one dictionary (color, pixels,
centroid, area, box) per detected block."""
    return []  # your code here
```

- `color`: the first word of the class name
- `pixels`: `result.masks.data`
- `box`: from `result.boxes.xyxy` to (left, top, width, height)



Which block is missing in the last frame? Is the estimate still right?

Get to work!

Summary

In this tutorial we

- replaced the God's-eye view with an RGB-D camera on the ceiling
- found the blocks by color (HSV, not RGB) and connected components
- turned pixels into 3D positions with the pinhole camera model and the depth image
- turned positions into facts with tracking, object permanence, the gripper's sensor and tolerances
- added sensor noise, kept a belief over facts, and filtered
- replaced the hand-written detector with an open-vocabulary detector, YOLOE

What comes next

What we left out

- orientations: full 6D poses
- where the robot itself is: SLAM
- where to look: active perception
- learning the predicates from examples

Next: Lab 3.2

Every piece is ready: a task planner, inverse kinematics, a motion planner, a controller, and a state estimator.

We put them together for the first time: task planning *then* motion planning.